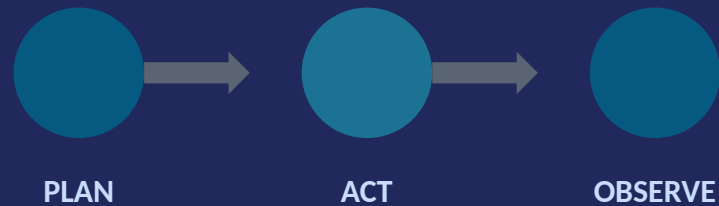


Agentic AI Workflows & Orchestration

Agent Decision Loops • Stopping Conditions • Multi-Tool Agents & Recovery



Agent Decision Loops

WHAT'S AHEAD

Multi-step task orchestration

Agent architectures: a planner that chooses which tool to call next

Decision loops

A Fixed Chain Can't Handle a Variable Number of Steps

PROBLEM

- A guest's free-text request may need one lookup, or several, depending on what the first one finds
- A fixed chain's graph shape is drawn once, before any guest ever sends a message
- Handling “maybe one step, maybe three” means pre-building every combination, or none
- The prior chain deliberately verified exactly one tool call, once

“Late checkout, and what does R-4471 say?”



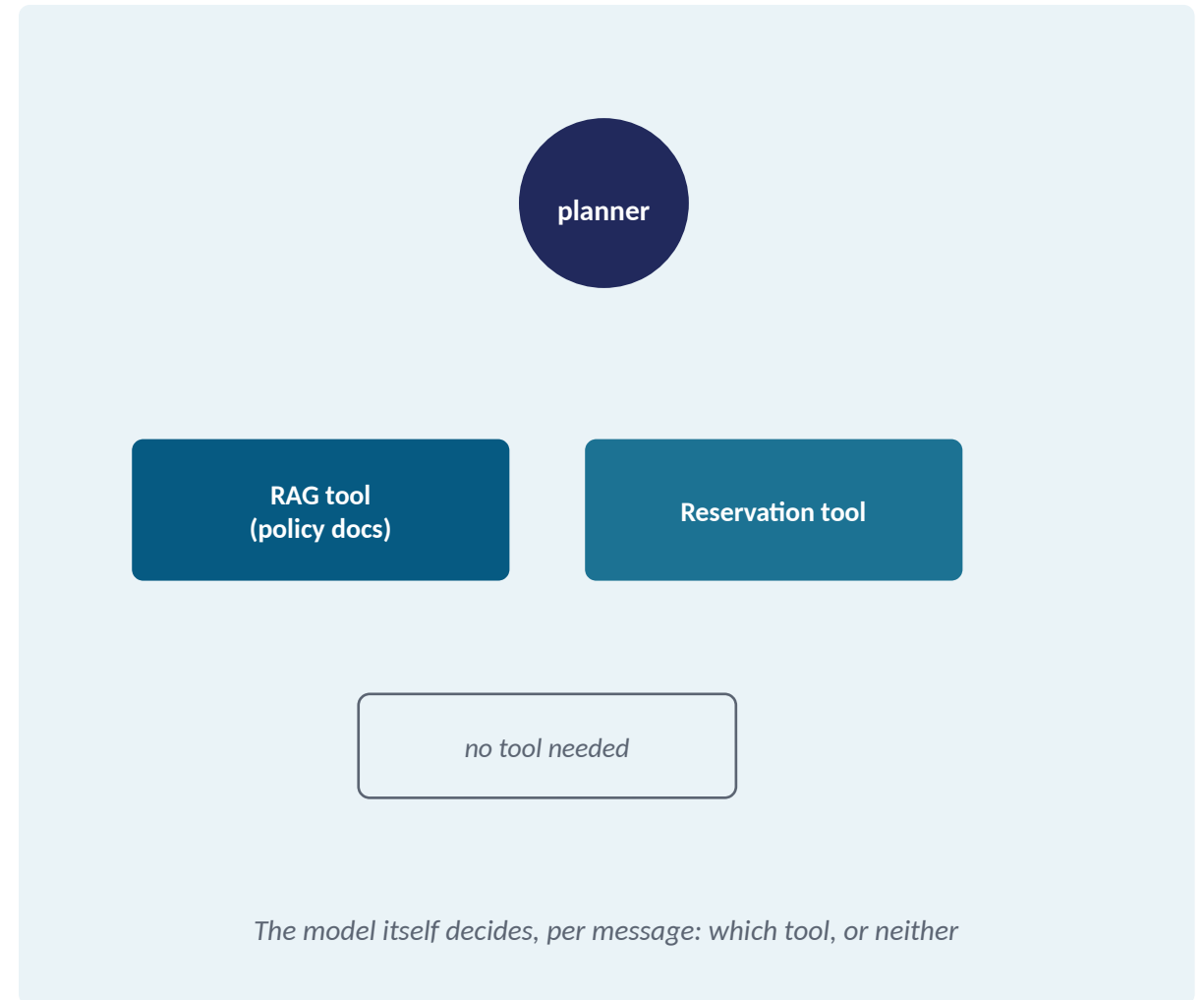
yesterday: one fixed path, drawn in advance

The number of steps a fixed chain takes is decided before any guest message arrives

Agent Architectures: A Planner That Chooses the Next Tool

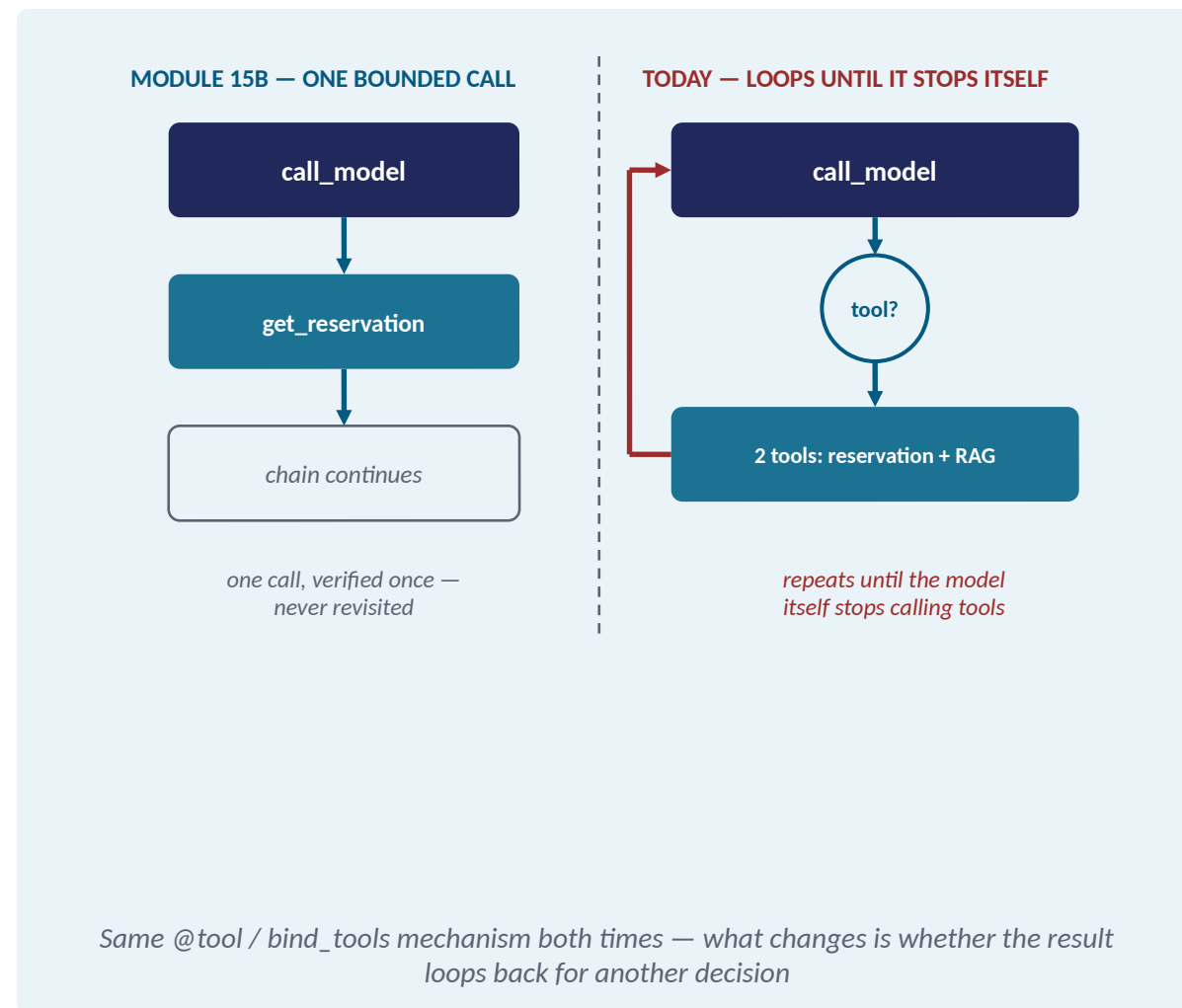
SOLUTION

- “Agent = Model + Harness” — the model supplies the judgment, the harness makes it safe to call repeatedly
- `create_agent`'s own definition: “a model calling tools in a loop until a given task is complete”
- Same `bind_tools(...)` mechanism as before — now with two real tools, not one
- Built by hand this time, so the routing decision is visible, not hidden inside a library call



One Bounded Tool Call vs. a Model That Loops on Its Own

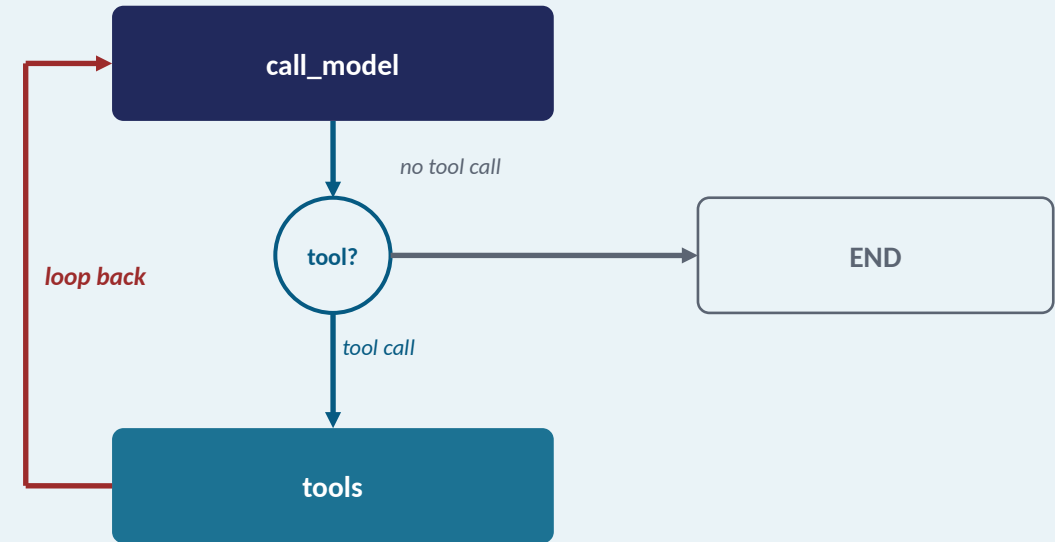
- Module 15B's tool call was one node, one tool, one call — fired once, verified once, never revisited
- The mechanism carries over completely unchanged: a type-hinted @tool function, bind_tools([...]), ToolNode running the real function and returning a ToolMessage
- What's new today: the tool node's result routes back to call_model instead of moving on, so the same model gets asked again
- Two tools now compete for that repeated decision, not one guaranteed call — the model, not the graph shape, decides how many laps it takes



Decision Loops: The One New Edge That Closes the Loop

SOLUTION

- Everything needed already exists: state, a node, a conditional edge
- `tools_condition`: routes to the tool node if the last message has tool calls, else ends
- The one new shape: the tool node's own edge points back to the model node
- That backward edge is the entire mechanical difference between a chain and an agent



Each lap feeds the previous tool's real result back into the next decision

HANDS-ON Design an agent's decision loop for a defined task, choosing a tool at each step; integrate Module 16B's RAG pipeline as one of the agent's callable tools.

Stopping Conditions, Multi-Tool Agents & Recovery

WHAT'S AHEAD

Stopping conditions for autonomous execution

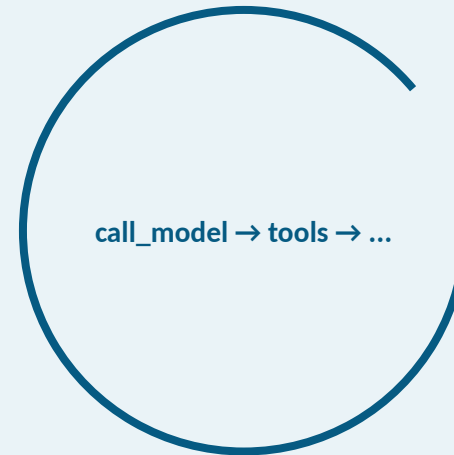
Multi-tool agents: how the model actually chooses

Retrying and recovering from a failed tool call

Nothing Stops the Loop From Running Forever

PROBLEM

- An ambiguous message, a confusing result, or a real model error can keep the loop going
- The loop built so far has no rule that can end execution the model wouldn't choose itself
- This is exactly the risk that kept the loop out of the last two modules until today
- Two different kinds of “stop” are needed, not one: completion, and a failsafe



no exit condition

A stopping condition was never optional — only deferred to exactly this point

Stopping Conditions: A Bounded Step Count Alongside Completion

SOLUTION

- Task completion: the model's own final answer, no further tool calls — already built
- `recursion_limit`: a graph-wide safety net, counts every super-step, hard error if exceeded
- A state-held `step_count`, checked in the same router, for a clean, designed stop
- Both stay set together — a designed bound, with a safety net underneath it

HANDS-ON Implement a stopping condition that halts the agent after a bounded number of steps or on task completion.

`step_count: 1 → 2 → 3 → STOP`

`MAX_STEPS` bound reached → clean end

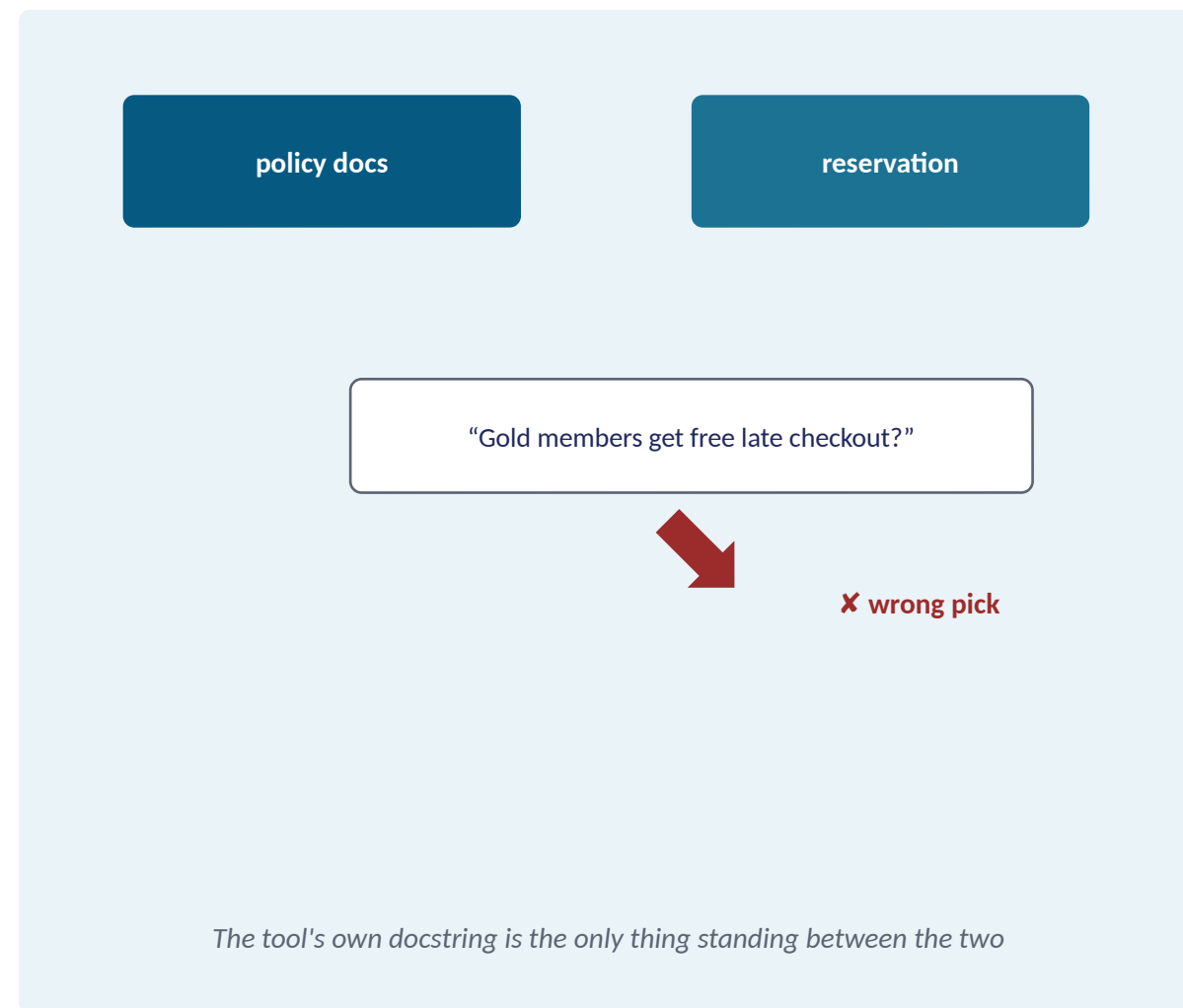
`recursion_limit` — underlying safety net

A designed, graceful bound — not the same as a hard `GraphRecursionError`

More Tools Means More Room for a Wrong Choice

PROBLEM

- With one tool, the model either calls it or doesn't — no wrong option to pick
- With two tools bound, a weak docstring can make the model confidently choose wrong
- A guest question can be genuinely ambiguous between “look this up” and “check my policy”
- Nothing about how a tool is called changes — the risk is entirely in which gets picked



Multi-Tool Agents: The Docstring Is the Whole Defense

SOLUTION

- Same `bind_tools([...])` mechanism, now doing double duty: call it at all, and pick correctly
- A docstring that says only what a tool does, not when, is the most common cause of a wrong pick
- Fix: state plainly what input the tool needs, and when not to use it
- Reused directly: the prior sprint's own `get_reservation` tool, now sharing the model's attention

HANDS-ON Give the agent a second callable tool alongside the RAG tool.

ONLY for a
named reservation

policy & amenities
questions

"Gold members get free late checkout?"

✓ correct pick

A clearer docstring, the same test message, a corrected choice

A Real Tool Call Can Simply Fail

PROBLEM

- Every tool call built so far has been demonstrated working, every time
- A real API request or database lookup can genuinely fail mid-call
- A bare ToolNode's default error handling only catches bad arguments, not real failures
- An agent meant to run unsupervised can no longer just let that crash the whole loop

`get_reservation(...)`



ConnectionError — call never returns

loop crashes — no result to feed back

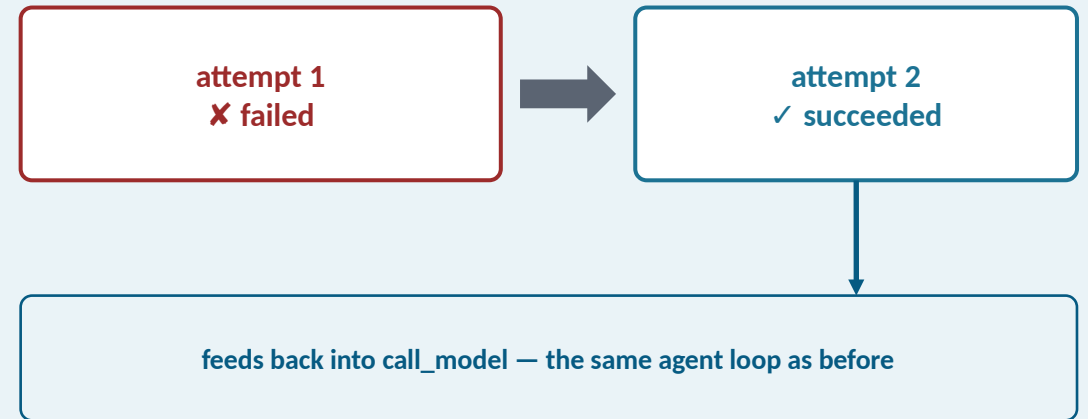
Deliberately narrow, exactly as the outline names it

Retrying and Recovering From a Failed Tool Call

SOLUTION

- The outline's own wording, made literal: reissue the identical failed call once, then give up
- A small wrapper: call, catch the exception, call again, only then return a failure message
- Not the same as a bare ToolNode's error handling — this catches real execution failures
- Production equivalent, named for comparison: ToolRetryMiddleware, configurable backoff

HANDS-ON Add a retry step that reissues a failed tool call once before giving up.



Recovered transparently — the model never sees the crash

Running a Test Task and Diagnosing a Failure Mode

- Every piece from today's objective is now built: a bound, two tools, a retry step
- Design one test message meant to expose a real failure, not one you know already works
- Read the step-by-step trace, not just the final answer, to see what actually happened
- Name the failure mode — a loop, or a wrong tool call — and fix the root cause

HANDS-ON Run the agent on a test task and diagnose one failure mode, a loop or a wrong tool call.

BEFORE

step 3: get_reservation
(wrong tool)

AFTER

step 3: answer_guest_
question (correct)

Diagnosing means reading the trace, not just noticing a wrong answer

Bonus: Workflow Patterns Beyond the Agent Loop

WHAT'S AHEAD

Routing: a hard-coded router vs. a model that picks the path

Parallelization: independent calls, run at once

Evaluator-optimizer: generate, score, and loop until it passes

Orchestrator-workers: one model decomposing its own task

Workflows vs. Agents: Four More Predefined Shapes

BONUS

- Anthropic's own distinction: a workflow runs through predefined code paths; an agent lets the model direct its own process
- Everything built in 17A and 17B is an agent — one model, deciding for itself, in an unbounded loop
- Module 15A's classify-then-draft chain was already a workflow: prompt chaining, the first of five named patterns
- Four more below, built from the same nodes and model calls — arranged differently because the task doesn't need a full loop

WORKFLOWS — predefined paths

Prompt chaining (Module 15A)

Routing

Parallelization

Evaluator-optimizer

Orchestrator-workers

AGENT — the model decides

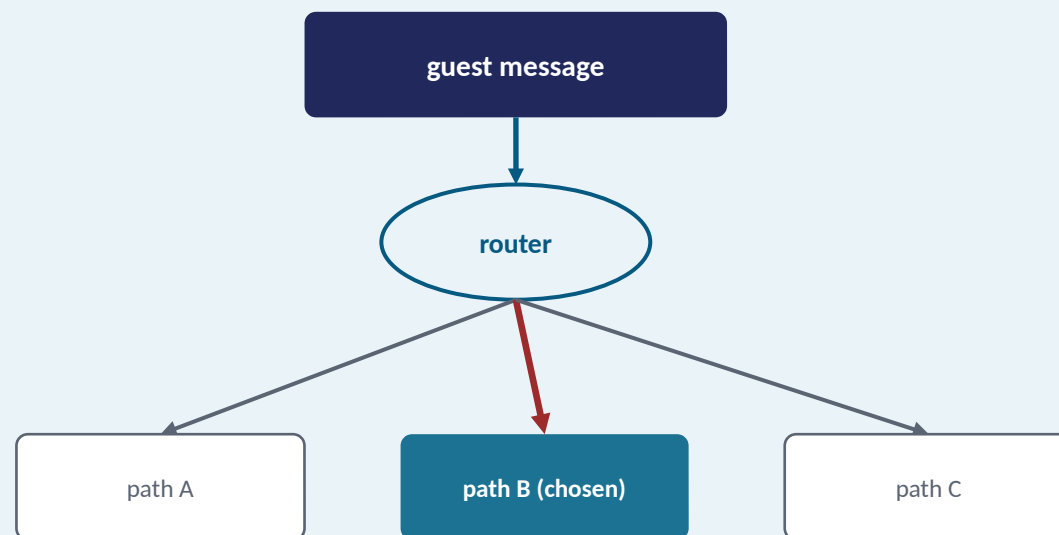
`call_model` ⇌ `tools` (today, 17A/17B)

Same building blocks throughout — a node, an edge, a model call — arranged for the task at hand

Routing: A Hard-Coded Router vs. a Model That Picks the Path

BONUS

- Module 15A's `route_by_category` already routes — but the routing function itself is plain Python, reading a category an earlier call produced
- A model-based router skips the separate classify step: the same `bind_tools([...])` choice from Module 17B's multi-tool slide, used once as a dispatch, not in a loop
- Anthropic's own fit: distinct categories, best handled by separate, tailored prompts
- Worth naming plainly — a learner who's only seen the hard-coded version can mistake an if/else for the whole pattern



Same dispatch idea either way — what changes is whether code or the model itself makes the pick

Parallelization: Independent Calls, Run at Once

BONUS

- Every call built so far has depended on the one before it — routing needs the category first, a retry needs the first attempt's result
- Two calls that don't depend on each other don't have to wait: `concurrent.futures` or `asyncio.gather` runs them together
- Built on the async surface Module 15's Foundations topic already introduced — `.ainvoke()` and `.astream()`
- Anthropic's own two shapes: sectioning (independent subtasks) and voting (the same task run several times for confidence)

SEQUENTIAL

call 1

call 2

longer total wait

PARALLEL

call 1

call 2

*both finish
together*

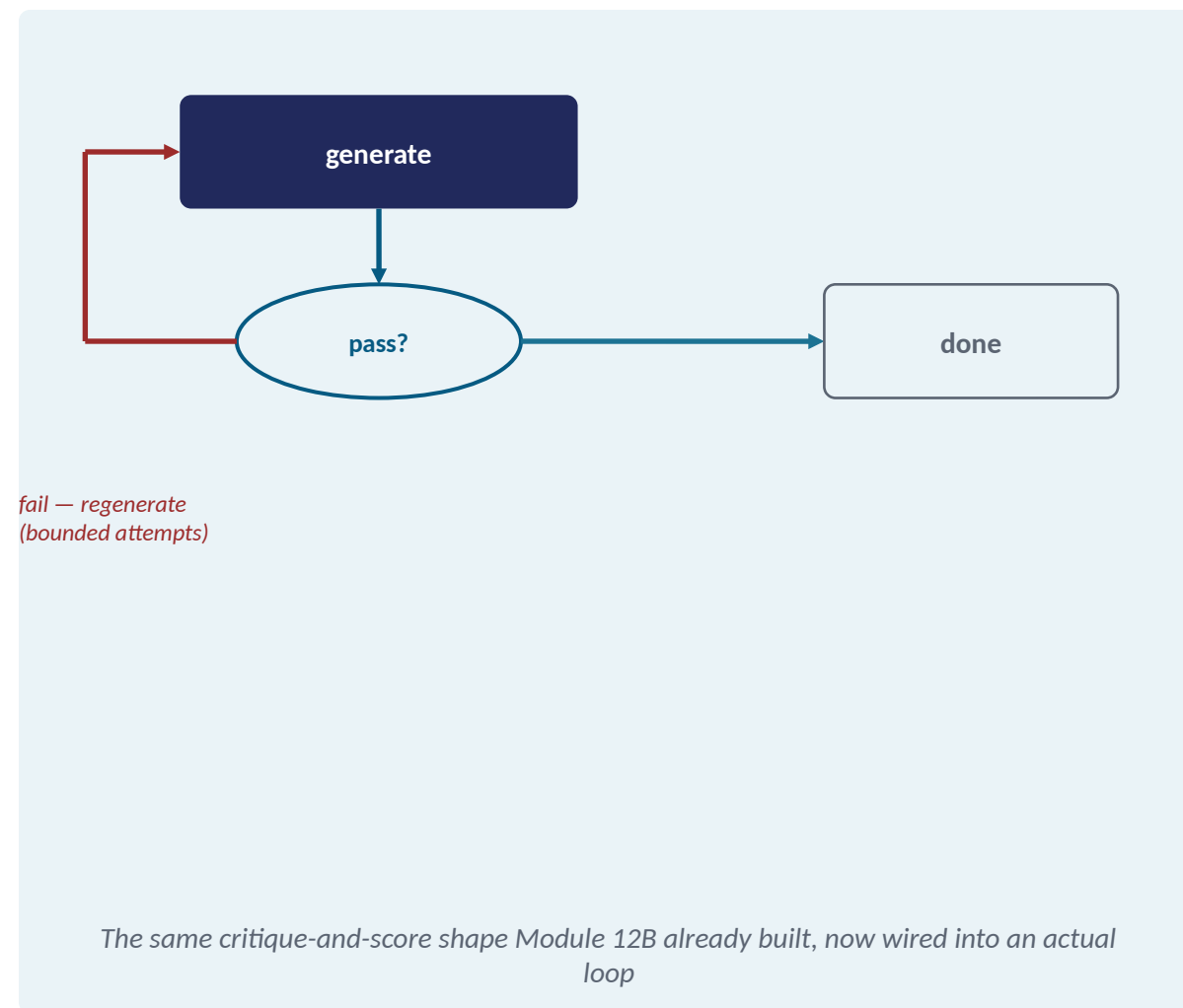
shorter total wait

Sectioning: two different subtasks at once. Voting: the same task run several times for a confidence check

Evaluator-Optimizer: Generate, Score, Loop Until It Passes

BONUS

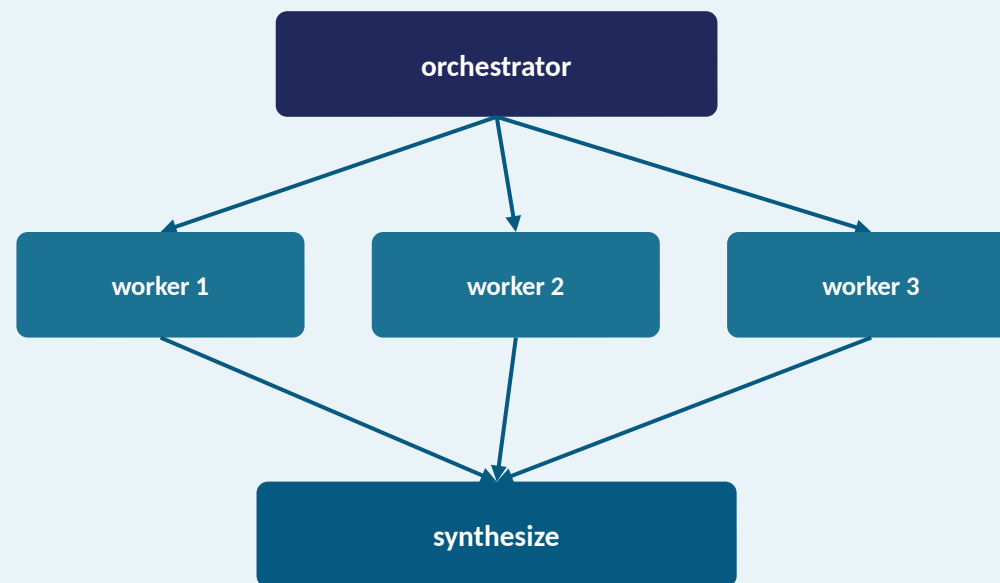
- Reuses Module 12B's own guardrail shape directly — a structured-output critique/score step, not a new mechanism
- One call generates a draft; a second, schema-scored call returns pass or fail, and why
- A failing draft goes back to the generator, informed by the evaluator's own stated reason — a second loop, bounded the same disciplined way as today's agent
- Anthropic's own condition for reaching for this: clear evaluation criteria, and evidence that feedback measurably improves the next draft



Orchestrator-Workers: One Model, Decomposing Its Own Task

BONUS

- Parallelization's subtasks are fixed in code; here the orchestrator call decides, itself, what the subtasks even are
- Each subtask dispatches to its own worker call; the orchestrator then synthesizes every result into one answer
- Explicitly one model in different roles at different steps — not a multi-agent system, the same line the day's own scope already drew
- Anthropic's own fit: tasks where the subtasks genuinely can't be predicted in advance



One model, called three different ways — decompose, do the work, synthesize — never more than one model coordinating